ProEm: The Line Codec

`src/core/line_codec.sv`

Paul Vincent Bezzina

17 October 2026

Abstract

The line codec is the part of each ProEm channel that touches the wire. On the way in it turns pin levels into bits and a bit clock (`tick`) for the frame chain; on the way out it turns the chain's entries into pin levels, with the right line code, timing, stuffing, start and stop conditions, and output drive. It is the one module where every standard's physical details meet, which is why it is the hardest to read. This document walks through it signal by signal: configuration, the receive path, the transmit state machine, the output stage, and the checks it reports. Names in `typewriter` are the signal names in the RTL, so the text can be read next to the code.

Contents

1 Where the codec sits **2**

2 Ports **2**

3 Configuration and derived signals **3**

4 Receive path **4**

4.1 Pin selection . . . 4
4.2 The three tick sources . . . 4
4.3 The bit timer (NRZ and NRZI) . . . 5
4.4 The Manchester decoder . . . 5
4.5 Bit value per line code . . . 6
4.6 The unstuffer . . . 6
4.7 What the chain receives . . . 7

5 Transmit path **7**

5.1 Entries in, `tx_done` out . . . 7
5.2 The state machine . . . 8
5.3 Packets . . . 8
5.4 Bit timing . . . 9
5.5 Shifting, stuffing and chaining entries . . . 10
5.6 From bit to level . . . 10
5.7 I^2C conditions . . . 10
5.8 End of packet . . . 11
5.9 Waiting on the receive grid . . . 11

6 Output stage **11**
6.1 Line and clock level by state 11
6.2 Pads 12

7 Checks reported to the chain **12**

8 Worked examples **12**
8.1 UART transmit, 115 200 baud 12
8.2 USB low speed receive 13
8.3 CAN acknowledge 13
8.4 I^2C controller write 13

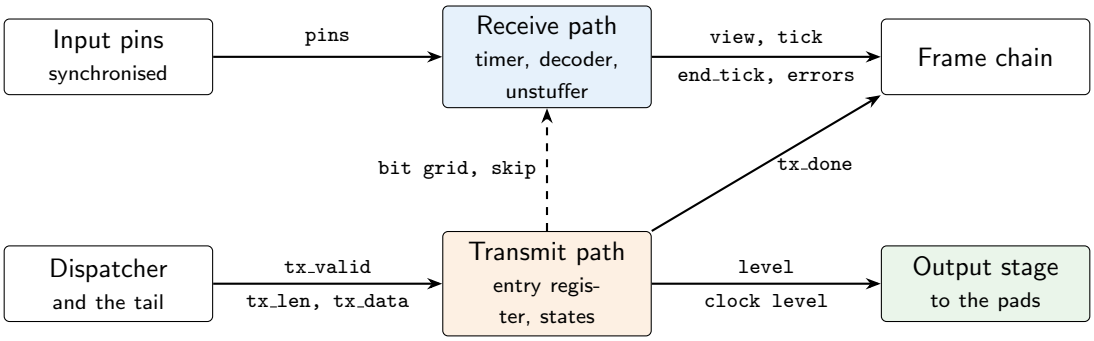
9 Limits and rules of thumb **13**

10 How it is tested **14**

A Internal signal index **14**

1 Where the codec sits

Each of the two channels has one codec, between the chip's pins and the channel's frame chain.



- **Receive.** Every input pin has already passed through two synchroniser flip-flops and one more register in `protocol_emulator`, so the codec sees clean, three-clock-late copies of the pads. It produces the pins the chain reads (`view`), one `tick` per bit, a second strobe for END (`end_tick`), and error and I^2C condition flags.
- **Transmit.** The dispatcher (and, since v0.6, the chain's tail for the CRC and reply) hands over *wire entries* of 0 to 8 bits. The codec holds one waiting entry, shifts bits out, and reports `tx_done` after each entry.
- **The two halves talk.** The transmitter uses the receive timer to align the reply gap, and the receiver resets its stuffing count when a transmission starts and ignores its own stuff errors while sending.

2 Ports

Port	Dir	Meaning
`pins[7:0]`	in	The eight protocol pads, synchronised.
`line_config[15:0]`	in	`LINE` instruction: code, inversion, pins, stuffing.
`clock_config[31:0]`	in	`CLOCK` instruction: tick source, edges, idle level, clock pin, `end_tick` select, divider.
`out_config[7:0]`	in	`OUT` instruction: drive mode, idle level, pair, end of packet, I^2C, select polarity.
`listen`	in	A read block wants a bit. With a generated clock this makes the codec run one clock period.
`line_free`	in	Every written bit of the frame has left the codec. Used for stuffing and the select output.
`in_frame`	in	The frame's START has matched or been sent. Unstuffing only arms inside a frame.
`frame_mark`	in	The frame's status entry reached the dispatcher: every bit of the frame has been handed over. Triggers STOP or the end of packet.
`halt`	in	`HALT` instruction; ends the current packet like a frame mark.
`tx_cancel`	in	Abort: drop the current and waiting entries and release the line.
`tx_valid`, `tx_len[3:0]`, `tx_data[7:0]`	in	A wire entry offered by the dispatcher or the tail.

Port	Dir	Meaning
tx_ready	out	The waiting-entry register is empty and no end of frame is pending.
tx_done	out	One pulse per entry finished (sent, skipped, waited or dropped).
view[7:0]	out	The pins as the chain sees them: PIN_A replaced by the decoded bit on a timed line.
tick	out	One pulse per bit for the chain's blocks.
end_tick	out	Strobe for the END block, selected by ETK.
code_error	out	A stuff bit had the wrong value (aborts the frame with a line error).
strict_tx	out	The line has fixed timing (not a generated clock), so running out of data is an underrun.
tx_mismatch	out	The pad read back differently from what the codec drove (arbitration).
i2c_target, i2c_start, i2c_stop	out	I^2C target mode is on; START or STOP seen on the bus.
data_out, data_oe	out	Data output and its enable.
pair_out, pair_oe	out	Complement output (USB D−) and its enable.
clock_out, clock_oe	out	Generated clock and its enable.
select_out	out	Chip select, active from START until the last bit has gone.

3 Configuration and derived signals

The three configuration words are registers in the control unit, written by LINE, CLOCK and OUT. The codec only slices them.

Word	Bits	RTL name	Instruction field
line_config	15:13	code	LINE CODE: 0 NRZ, 1 NRZI, 2 NRZI pair, 3 Manchester, 4 Manchester Thomas
	12	inv	INV
	11	boundary	BND: Manchester, first edge after idle is a bit boundary (v0.7)
	10:8	pinA	PIN_A, the data pin
	7	stuffAny	SANY: stuffing counts runs of either value
	6:3	stuffLen	STUFF: run length, 0 = off
	2:0	pinB	PIN_B, second wire of a pair
clock_config	31:30	source	SRC: 0 SYNC, 1 ASYNC, 2 GENERATE, 3 RECOVER
	29:28	edgeSel	EDGE: 0 rising, 1 falling, 2 both
	27	clockIdle	IDLE: generated clock level between frames
	26:24	clockPin	PIN: clock input, or the read-back of a generated clock
	23:22	endSel	ETK: what end_tick is
	21:16, 15:0	div	DIV, 22 bits (v0.7): clocks per bit (timed) or per half period (generated)
out_config	7:6	drive	DRIVE: 0 off, 1 push-pull while sending, 2 open drain, 3 push-pull always
	5	outIdle	IDLE: data level between frames
	4	pairEnable	PAIR: drive the complement output
	3	eopEnable	EOP: end of packet at the end of each frame
	2	i2c	I2C: I^2C conditions
	1	selectHigh	CSH: select output active high

From these the codec derives a set of one-bit modes that the rest of the logic uses instead of the raw

fields:

Signal	Definition	Meaning
`timed`	`source` = ASYNC or RECOVER	The codec's own bit timer sets the bit grid.
`generating`	`source` = GENERATE	The codec drives the clock output.
`manchester`	`code` = 3 or 4	Manchester decode and encode.
`thomas`	`code` = 4	Opposite Manchester convention.
`nrzi`	`code` = 1 or 2	NRZI: no change is 1, change is 0.
`pairCode`	`code` = 2	NRZI on a differential pair; SE0 and SE1 detected.
`openDrain`	`drive` = 2	Drive low for 0, release for 1, on data and clock.
`i2cTarget`	`i2c` and `source` = SYNC	I^{2}C target: SCL is an input.
`half`	`div` $\gg 1$	Half a bit. All the timers are 22 bits wide, like `div`, so the slowest bit is 4 194 303 clocks (84 ms at 50 MHz).
`blank`	$\lfloor 3\,$`div`$/4 \rfloor$	Manchester edge blanking window.
`twoBits`	$2 \times$ `div`	Manchester lock timeout, and the length of the end of packet.

4 Receive path

4.1 Pin selection

```
rawA    = pins[pinA]          the data pad as it is
lineA   = rawA ^ inv          the data line in "logical" polarity
lineB   = pins[pinB]          second wire of a pair (USB D-)
clockIn = pins[clockPin]      the clock pad (SYNC input, or GENERATE read-back)
```

Each signal also has a one-clock-old copy (`prevA`, `prevRaw`, `prevClock`), which gives the edge detectors: `aEdge` (any change of `lineA`), `clockRise`, `clockFall`. On a pair, `se0` is both wires low and `se1` both high. Note that the SE0 and SE1 tests use `rawA`, not `lineA`: they are about the physical wires, whatever the polarity.

4.2 The three tick sources

The chain's blocks take one bit per `tick`. Where the tick comes from is the single biggest switch in the codec:

Source	`tick` =	How
SYNC	`syncTick`	An edge of `clockIn`: rising, falling or both (`edgeSel` 0, 1, 2). The chain reads the raw pins on that edge. Used for SPI target, PS/2, I^{2}C target, a looped-back clock.
ASYNC, RECOVER	`rxTick`	The receive bit timer below: one tick per decoded, non-stuff bit, with the decoded bit placed on `PIN_A` of `view`. Used for UART, USB, CAN, Manchester.
GENERATE	`genTick`	The transmit state machine, on the sampling edge of the clock it is generating (Section 5.4). Used for SPI controller and I^{2}C controller.

`view` is the raw `pins` except on a timed line, where `view[pinA]` is replaced by `rxBit`, the last decoded bit. END normally reads another pin or uses the pair compare, so it still sees the real line; `end_tick`

gives it a strobe of its own: `endSel` 0 is the same as `tick`, 1 is every system clock, and 2 is `rxStrobe`, the mid-bit sample strobe, which keeps running through stuff bits and SE0 (USB needs that to count two bit times of SE0).

4.3 The bit timer (NRZ and NRZI)

For a timed, non-Manchester line, `rxCount` is a counter that measures position inside the current bit:

```
on any edge of lineA       rxCount <= 1          (re-centre)
else if rxCount >= div-1   rxCount <= 0          (next bit boundary)
else                       rxCount <= rxCount + 1
strobe = (rxCount == half)                       (sample in the middle)
```

An edge always marks a bit boundary, so the timer restarts there, and the sample is taken half a bit later. Between edges (a run of equal bits) the timer free-runs one bit per `div` clocks. This is the same trick a UART receiver uses, applied to every bit instead of only the start bit, so the receiver follows a sender whose clock is a few percent off for as long as edges keep coming. Bit stuffing exists precisely to guarantee those edges.

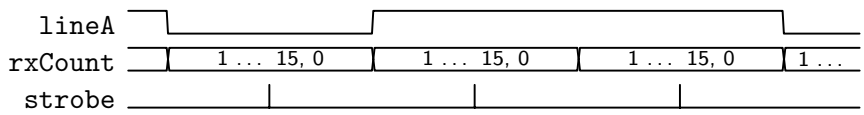


Figure 1: Bit timer with `div` = 16: the edge restarts the count, each strobe is 8 clocks after a boundary.

4.4 The Manchester decoder

A Manchester bit always has an edge in its middle, and has an extra edge at its boundary when two equal bits follow each other. The decoder must use the middle edges and ignore the boundary ones. It does that with a blanking window instead of a sample point:

```
strobe = aEdge && (locked ? rxCount > blank : !boundary)     blank = 3/4 of a bit
on strobe:   rxCount <= 1, locked <= 1
first edge after idle with BND set:  rxCount <= half, locked <= 1, no bit
otherwise:   rxCount counts up (saturating); locked <= 0 after two bit times
bit value  = lineA ^ thomas                          the level after the edge
```

After an accepted (middle) edge, any edge within $\frac{3}{4}$ of a bit is a boundary edge and is ignored; the next middle edge comes one bit later, well past the window. If the line goes quiet for two bit times, `locked` drops and the next edge is accepted at once, which is how the decoder catches the first edge of a frame. With IEEE convention a low-to-high middle edge is a 1, so the level after the edge is the bit; Thomas convention inverts it.

The first edge after idle (v0.7). Taking the first edge as a middle edge is right when a frame begins with a preamble (10BASE-T) or with a bit whose first half equals the idle level (RC5). It is wrong when the first bit's first half differs from idle, as in DALI, whose start bit falls from the idle-high bus at the bit boundary: the decoder would then lock half a bit out of phase. With `BND` set, the first edge after idle produces no bit; it sets `rxCount` to half a bit and `locked`, so the middle edge half a bit later sees `rxCount` at a full bit, passes the window, and is decoded as the first bit. One edge after idle cannot tell the two cases apart by timing alone, because a boundary edge also follows a middle edge whenever two equal bits are sent, which is why this is a setting rather than automatic.

The window must sit strictly between half a bit (a boundary edge) and one bit (the next middle edge), with room for jitter. At 4 clocks per bit, $\frac{3}{4}$ is 3 clocks and a boundary edge 2 clocks after a middle edge is too close to call once jitter is added, so 5 clocks per bit is the minimum (10BASE-T at 50 MHz).

4.5 Bit value per line code

Code	`bitValue`	Notes
NRZ	`lineA`	The sampled level.
NRZI, NRZI pair	`lineA == lastSample`	Same level as the last sample is a 1, a change is a 0. `lastSample` is updated on every strobe, and forced to 0 during SE0 or SE1 so the first bit after an end of packet is measured from J.
Manchester	`lineA ^ thomas`	Level after the accepted middle edge.

`dataValid` is a strobe that is not SE0 or SE1: on a pair, those states carry no data bits. They still produce `rxStrobe`, for END.

4.6 The unstuffer

When `stuffLen` is not zero, the transmitter has inserted one extra bit after every run of `stuffLen` equal bits, and the receiver must take it out again without giving it a tick. Two styles exist:

Standard	`stuffLen`	`stuffAny`	What counts as a run
USB	6	0	Consecutive 1s only. The stuff bit is a 0 (in NRZI, a forced transition).
CAN	5	1	Consecutive equal bits, 0s or 1s. The stuff bit is the opposite value.

The state is three registers: `runCount` (length of the current run), `runValue` (its value, for `stuffAny`) and `skipNext` (the next data bit is a stuff bit).

```
runMatch = stuffAny ? (bitValue == runValue) : bitValue
nextRun  = runMatch ? runCount + 1 : (stuffAny ? 1 : 0)

dropBit  = dataValid && stuffOn && skipNext         this bit is a stuff bit
goodBit  = dataValid && !dropBit                    this bit goes to the chain
stuffBad = dropBit && (stuffAny ? bitValue == runValue : bitValue)
```

- On a good bit, the run is extended or restarted, and `skipNext` is set when the run reaches `stuffLen`.
- On a dropped bit, `skipNext` clears and a new run starts: with `stuffAny` the stuff bit itself is the first bit of the next run (CAN counts it), without it the count restarts at 0 (USB's stuff bit is a 0 and so ends a run of 1s).
- A stuff bit with the wrong value (`stuffBad`) is registered as `code_error`, which aborts the frame with a line error. USB uses this as an error check; CAN calls it a stuff error.
- The run is cleared on SE0 or SE1 (USB end of packet) and when a transmission starts (`txBegin`), so a packet never inherits a run from before it.

Three conditions guard `skipNext` and `code_error`. They were each added after a test failed, so they are worth knowing:

1. **Arm only inside a frame** (`in_frame`). A CAN bus idles recessive (1). Without this, five idle 1s armed the skip and the start-of-frame 0 was thrown away as a stuff bit.
2. **Arm only when the codec is sending a packet or nothing is queued** (`packetActive || line_free`). In a transmit frame, `in_frame` goes high as soon as START is queued, before the first bit is on the wire; idle bits seen in that gap must not count.
3. **Ignore stuff errors while sending** (`!packetActive`). When the codec is driving a line whose pad is not read back on `PIN_A`, the decoder sees a static line and would report errors in the chip's own transmission.

4.7 What the chain receives

`rxBit` and `rxTick` are registered from `bitValue` and `goodBit`, so the chain sees each bit one clock after its strobe, and the bit and its tick arrive together. A stuff bit produces neither. The chain's CRC unit is fed from the same tick, which is why the CRC never includes stuff bits.

5 Transmit path

5.1 Entries in, `tx_done` out

The codec holds at most one waiting entry, in `nextValid`, `nextLen` and `nextData`, plus the one it is shifting out (`shiftData`, `shiftLeft`). It accepts a new one whenever the waiting slot is empty:

```
tx_ready = !nextValid && !endPending && !tx_cancel
```

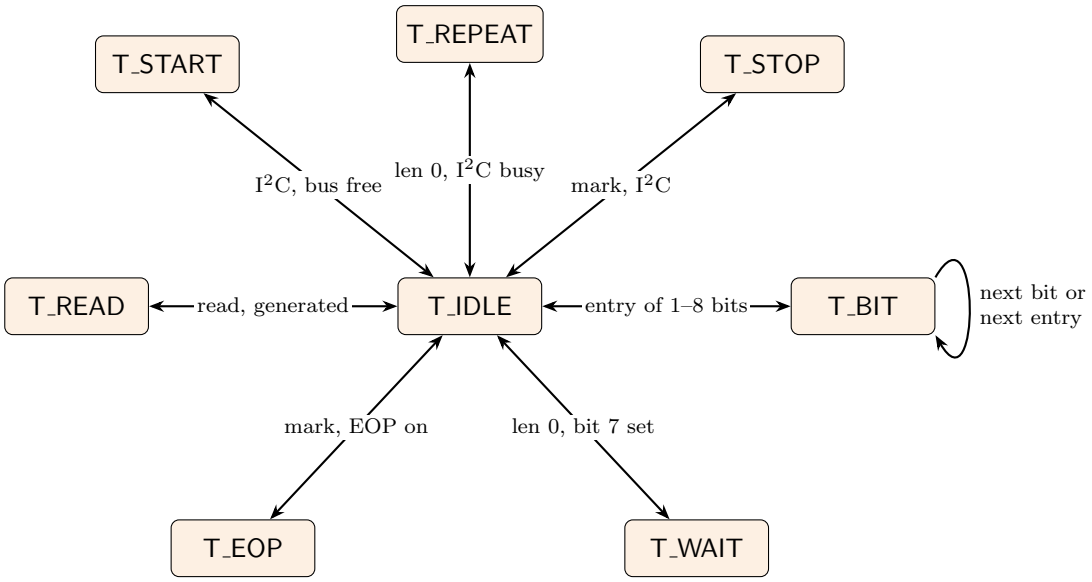
Bits leave from bit 0 upward. MSB-first standards are handled by the chain, which reverses each field before it reaches the codec, so the codec never needs to know the bit order. An entry's length is 1 to 8 bits; length 0 is special:

Entry	Condition	Meaning
length 0, `data[7]` = 1	timed, not Manchester	Wait `data[6:0]` bit times without driving, aligned to the receive bit grid (`T_WAIT`). Used for the reply gap.
length 0	I^2C controller, bus busy	Repeated START (`T_REPEAT`).
length 0	otherwise	Nothing to send; counts as done at once.

Every entry ends with exactly one `tx_done` pulse, whether it was sent, waited, skipped or dropped by a cancel. The chain counts entries in flight with it and only lets read blocks listen once that count is zero (`line_free`).

`endPending` is set by `frame_mark` or `halt`. It stops new entries being taken until the end-of-frame action (STOP or end of packet) has been done, so the next frame's bits can never slip in before it.

5.2 The state machine



Every state returns to `T_IDLE` when it is done; `T_BIT` stays as long as bits, stuff bits or a waiting entry keep coming.

`T_IDLE` decides what to do next, in this order of priority:

1. A waiting length-0 entry: start a wait, a repeated START, or simply finish it.
2. A waiting entry on an idle I^{2}C controller bus: send START first (`T_START`); the entry is sent afterwards.
3. A waiting entry, and (I^{2}C target only) SCL has just fallen: load it and send (`T_BIT`). This is `txBegin` when no packet is active.
4. Generated clock, a read block listening, no end pending: run one clock period with the line released (`T_READ`).
5. An end pending: STOP on an I^{2}C controller (`T_STOP`), end of packet if enabled (`T_EOP`), otherwise just close the packet.

A `tx_cancel` overrides everything: it goes to `T_IDLE`, clears the packet, I^{2}C and stuffing state, and adds to `dropCount` the number of entries it threw away (the one being shifted, if any, plus the waiting one), so `tx_done` is still pulsed once for each over the next clocks. While `dropCount` is not zero the state machine waits in `T_IDLE`, so no other `tx_done` can fall on the same clock, and a second cancel adds to the count instead of overwriting it. (Before 13 October 2026 a second cancel within two clocks, or a dropped entry answered on the same clock as the next entry, lost one `tx_done`; the chain then waited for it forever. The formal proof in `formal/line_codec_formal.sv` found it.)

5.3 Packets

`packetActive` is set when the first entry of a frame starts and cleared at the end of packet, STOP, or the end-of-frame action. While it is set:

- the NRZI level and the transmit stuffing run carry on from one entry to the next, so a frame made of many entries is one continuous coded stream;
- between entries, when the next one has not arrived yet, the line holds the last level sent (`holdLevel`) instead of going to the idle level. An I^{2}C target with open drain is the exception: it releases SDA.

When the first entry of a packet loads, the transmit run is cleared and the NRZI level is set as if the line had been at J: `nrziLevel = !data[0]`.

5.4 Bit timing

`T_BIT` sends one bit at a time; `txCount` measures time inside the bit, and `endOfBit` says when to move on. It depends on the tick source:

Source	Bit timing
Timed	One bit every `div` clocks: `endOfBit` at `txCount = div − 1`. The transmit grid is free-running and independent of the receive timer.
SYNC	The output changes on the clock edge opposite the sampling edge (`shiftEdge`): falling if the chain samples on rising, and rising if it samples on falling. Used for SPI target and I^{2}C target.
Generated	Two phases of `div` clocks each. Phase A (`phaseB = 0`) holds the clock at `phaseAClock`, its idle level (forced low for I^{2}C); phase B holds the opposite level. The data changes at the start of phase A, so it is stable across the A-to-B edge.

Generated clock details.

- `sampleLeading` says whether the chain samples on the A-to-B edge (the middle of the bit) or on the B-to-A edge (the end). It is true when `edgeSel` is "both", or when the chosen edge is the one that leaves the idle level. SPI modes 0 and 2 (CPHA 0) sample on the leading edge; modes 1 and 3 (CPHA 1) on the trailing one. `genTick` pulses on the chosen edge, which is how the chain gets its tick in this mode.
- With a trailing-edge sample (SPI CPHA 1) the data pin keeps the previous bit (`holdLevel`) through phase A and changes to the new bit on the leading edge, so it is steady for a whole phase on either side of the device's sampling edge. Before 13 October 2026 it changed at the start of phase A, on the same clock as the device's sampling edge, which leaves no hold time.
- Clock stretching: in phase B, with open drain, the codec has released the clock and waits until the clock read-back (`clockIn`, the clock pin) is actually high (`clockHigh`) before it starts counting. A slow I^{2}C device holding SCL low therefore just lengthens the bit.
- The clock only runs while there is something to do: a bit to send (`T_BIT`) or a read block listening (`T_READ`). Between bytes on I^{2}C the controller holds SCL low.
- Where the sample falls. `genTick` is registered and the input pins are three clocks late, so with a push-pull clock the chain sees the line as it was about one system clock before the chip's sampling edge, which suits any device that changes its output on or after that edge (MDIO, SPI). With an open-drain clock the codec first waits for the read-back to show the clock high, so the sample is taken just after the edge; I^{2}C needs this because a target may change SDA while it stretches SCL.

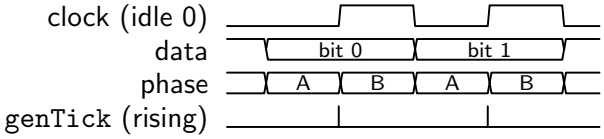


Figure 2: Generated clock, SPI mode 0: `div` clocks per phase, data set at the start of phase A, sampled on the rising A-to-B edge.

5.5 Shifting, stuffing and chaining entries

At `endOfBit` the machine decides the next bit. The combinational signals computed for that moment are:

```
dataBitEnded = !stuffing                    the bit just sent was data, not a stuff bit
txRunNew     = run length including it      (stuffAny: equal bits; else 1s)
willStuff    = dataBitEnded && stuffOn && txRunNew == stuffLen
stuffValue   = stuffAny ? !sendBit : 0
leftAfter    = shiftLeft - (dataBitEnded ? 1 : 0)
entryDone    = !willStuff && leftAfter == 0
continueNext = entryDone && nextValid && nextLen != 0
nextBitValue = willStuff ? stuffValue : (entryDone ? nextData[0] : dataAfter[0])
```

- A stuff bit is sent *instead of* advancing the shift register: `stuffing` is set, `sendBit` becomes `stuffValue`, and the data stays where it was. After the stuff bit, the run restarts (at 1 with `stuffAny`, since CAN counts the stuff bit; at 0 for USB).
- An entry is not done while a stuff bit is still owed after its last bit. That is why a CAN CRC that ends in five equal bits is followed by its stuff bit before the delimiter.
- If the next entry is already waiting when the current one ends, its first bit follows with no gap (`continueNext`). If not, the machine returns to `T_IDLE`, the line holds its level, and the next entry starts a fresh bit period when it arrives. On a timed line that pause shows up as a longer bit, which is why the chain keeps the codec fed and reports an underrun if it cannot.
- `tx_done` pulses when `entryDone`.

5.6 From bit to level

Code	`bitLevel`
NRZ	`sendBit`.
NRZI	`nrziLevel`, which toggles for every 0 and holds for every 1 (updated together with `sendBit`).
Manchester	On a timed line, `!sendBit` for the first half of the bit and `sendBit` for the second (XOR `thomas`), so a 1 is a low-to-high middle edge. `div` must be even.

`level` is then chosen by state (Section 6), and the pad value is `dataPin` = `level ^ inv`, or 0 during SE0.

5.7 I²C conditions

On an I²C controller (generated clock with `i2c`), three states draw the bus conditions, each in steps of `div` clocks. In steps that raise SCL with open drain, the step waits until SCL really is high, so stretching works here too.

State	Step 0	Step 1	Step 2	Step 3	Result
`T_START`	SDA 0, SCL 1	SDA 0, SCL 0			SDA falls while SCL is high
`T_REPEAT`	SDA 1, SCL 0	SDA 1, SCL 1	SDA 0, SCL 1	SDA 0, SCL 0	repeated START
`T_STOP`	SDA 0, SCL 0	SDA 0, SCL 1	SDA 1, SCL 1		SDA rises while SCL is high

`i2cBusy` is set after START and cleared after STOP. While it is set, the idle state holds SCL low. As a target (SYNC clock on SCL), the codec does not draw conditions; it detects them:

```
i2c_start = SDA falls (prevRaw && !rawA) while SCL is high now and last clock
i2c_stop  = SDA rises (!prevRaw && rawA) while SCL is high now and last clock
```

and waits for SCL to fall (`startWait`) before driving each new bit, so its data meets the bus's hold and set-up rules.

5.8 End of packet

When the end is pending, a packet is active and `eop` is set, `T_EOP` sends:

- on a pair (USB): two bit times of SE0 (both outputs low, `se0Out`), then one bit time of J, then the line is released;
- on a single line (10BASE-T): two bit times high.

The NRZI level is reset afterwards, so the next packet starts from J again.

5.9 Waiting on the receive grid

`T_WAIT` implements the wait entry used for the reply gap. It does not drive anything (the level is the idle level), and works in two parts:

1. **Align.** Wait until the receive timer reaches a bit boundary (`rxCount` $\geq$ `div` $-$ 1). If the unstuffer has a stuff bit pending at that moment (`skipNext`), add one bit to the wait: that bit belongs to the frame just received (CAN after a CRC ending in five equal bits).
2. **Count.** Count `waitBits` bit times of `div` clocks, then finish the entry.

The reply that follows therefore starts exactly on the bit grid of the frame it answers, which is what a CAN ACK slot demands.

6 Output stage

6.1 Line and clock level by state

State	`level` (data)	`clockLevel`
`T_IDLE`, `T_WAIT`	`idleLevel`	I^2C: low while busy, high when free. Otherwise `clockIdle`.
`T_BIT`	`bitLevel`, see below	phase A: `phaseAClock`; phase B: the opposite
`T_READ`	released (1 open drain, else `outIdle`)	as `T_BIT`
`T_START`, `T_REPEAT`, `T_STOP`	per step (table above)	per step
`T_EOP`	0 on a pair (SE0, then J), 1 otherwise	`clockIdle`

In `T_BIT`, phase A of a generated clock that samples on the trailing edge (and is not I^2C) shows `holdLevel`, the previous bit, instead of `bitLevel`, so the data changes on the leading edge.

`idleLevel` is `holdLevel` inside a packet (except an I^2C target with open drain), otherwise 1 with open drain or `outIdle` with push-pull.

6.2 Pads

Output	Value	Enable
data	push-pull: `dataPin`; open drain: 0	push-pull always (`drive` 3), or while `driveNow` (`drive` 1); open drain: when `dataPin` is 0
pair	`!(level ^ inv)`, or 0 during SE0	as push-pull data, if `pair` is set
clock	push-pull: `clockLevel`; open drain: 0	generated clock and `drive` not off; open drain: when `clockLevel` is 0
select	`selectActive`, inverted unless `csH`	no enable (a plain output)

`sending` is any state other than idle and wait, or an active packet. `driveNow` (v0.7) is `sending` except in `T_READ` and, with a generated clock, in `T_IDLE`: a "push-pull while sending" output lets go of the line for read bits, so a device can answer on the same wire (3-wire SPI, MDIO). Timed lines still hold their level between entries of a packet. `selectActive` covers the whole frame: from START (`in_frame`) through every queued and in-flight bit (`nextValid`, `!line_free`), so SPI chip select never glitches between entries. With `drive` = 0, after reset or `CLEAR`, nothing but select is driven.

7 Checks reported to the chain

Read-back mismatch. While sending a bit, the codec compares the pad it drives with the pad it reads back on `PIN_A`, at a `checkPoint`:

Source	Check point
Timed	middle of the bit (a quarter bit for Manchester, before the middle edge)
Generated	middle of phase B, once the clock is high
SYNC	the sampling edge

`tx_mismatch` = `T_BIT` and check point and `rawA` ≠ `dataPin`. The branch unit uses it for CAN-style arbitration: the chip sent recessive and read dominant, so another node won. Because the input pipeline is three clocks long, a timed line needs at least 8 clocks per bit for the check to land inside the bit.

Stuff error. `code_error`, described with the unstuffer.

Strict timing. `strict_tx` is high for every source except a generated clock: with a generated clock the codec can simply pause, but on a timed or externally clocked line, running out of data mid-frame breaks the frame, so the chain reports an underrun.

8 Worked examples

8.1 UART transmit, 115 200 baud

`CLOCK` ASYNC, `div` = 434; `LINE` NRZ; `OUT` push-pull always, idle 1. The chain sends a 1-bit entry {0} (start bit) from START, then entries of 8 data bits and {1} (stop bit) from MAIN. The codec takes the start entry, holds 0 for 434 clocks, chains straight into the data entry (it was already waiting), sends its 8 bits LSB first at 434 clocks each, then the stop bit. At the frame mark there is no end of packet, so the packet just closes and the line returns to `outIdle` = 1.

8.2 USB low speed receive

`LINE` NRZI pair, `stuffLen` = 6, `stuffAny` = 0; `CLOCK` RECOVER, `div` = 33. Each edge on D+ restarts `rxCount`; the strobe 16 clocks later samples J or K and compares with the last sample. After six 1s (six bit times without an edge) `skipNext` is set and the next strobe's bit, which must be a 0 (a transition), is dropped without a tick. The two bits of SE0 at the end produce strobes but no data, clear the run, and are counted by END through `end_tick` = `rxStrobe`.

8.3 CAN acknowledge

`LINE` NRZ with `stuffLen` = 5, `stuffAny` = 1; open drain; `CLOCK` ASYNC. As the last CRC bit is sampled, the chain's tail queues a wait entry with `GAP` = 0 and a 2-bit reply {1, 0}. `T_WAIT` waits for the next receive bit boundary, adds one bit if a stuff bit is owed, and finishes. The reply then sends a 1 (released, the CRC delimiter) and a 0 (driven low) exactly in the ACK slot.

8.4 I²C controller write

`CLOCK` GENERATE, `div` = 250 (100 kHz); `OUT` open drain with `i2c`. The first entry finds the bus free: `T_START` pulls SDA low with SCL high, then SCL low. The address and data bits go out in `T_BIT`, SDA changing while SCL is low and sampled on the rising edge; the ACK bit is a read, run by `T_READ` with SDA released. At the frame mark, `T_STOP` raises SCL and then SDA, and the bus is free again.

9 Limits and rules of thumb

- **Manchester receive** needs at least 5 clocks per bit; **Manchester transmit** needs an even `div`.
- **Generated clocks** need `div` $\geq$ 4 clocks per half period.
- **Read-back on a timed line** needs at least 8 clocks per bit (three clocks of input pipeline).
- **Seamless transmit** needs the next entry waiting before the current one ends; otherwise the line pauses for at least one clock and a timed bit is stretched.
- **The wait entry** needs a timed, non-Manchester line, because it aligns to the receive bit timer; elsewhere it is skipped.
- **One data pin per codec.** `PIN_A` is the only decoded pin; other pins are seen raw.
- **Bit rate range.** With the 22-bit `div`, from 12 bit/s up to the per-code minimums above, at 50 MHz.

10 How it is tested

Codec feature	Tests
Bit timer, NRZ, re-centring	`t_uart` (sender 2 % fast and slow), `t_stream`, cocotb UART tests
NRZI pair, SE0/SE1, unstuffing, end of packet	`t_usb`, `t_crc` (USB)
Manchester decode and encode	`t_ethernet` (jittered line, 50 and 60 MHz), `t_manchester`
Stuffing with `stuffAny`, wait entry	`t_crc` (CAN from a node model and chip to chip)
SYNC source, `shiftEdge`	`t_spi` (target), `t_ps2`, `t_i2c_target`
Generated clock, stretching, I²C conditions, `T_READ`	`t_i2c`, `t_spi` (controller), cocotb SPI and I²C loopbacks
I²C target detection	`t_i2c_target`, cocotb I²C loopback
Generated clock in all four SPI modes, trailing-edge sampling	`t_modes` (device model with no hold margin)
Read-back mismatch, cancel	`t_branch` (CAN-style arbitration), `t_stream` (underrun)
Random programs through the codec	`t_random`, `t_random_branch`
Entry accounting, cancels, output enables	`formal/line_codec.sby`: unbounded proof with every configuration free

The v0.7 additions are tested in `t_standards`: the wide divider by RC5 (88 900 clocks per bit) and the K-Line 5-baud init, `BND` by DALI, release-while-reading by 3-wire SPI and MDIO with checks that the chip and the device never drive at the same time. Not yet exercised by a test: single-ended NRZI (code 1) and Thomas Manchester (code 4). They use the same paths as the tested codes with one gate different. The formal proof covers the transmit side's entry accounting and the output enables; the bit-level timing (the 22-bit timer, the Manchester decoder and the unstuffer) is checked by simulation only.

A Internal signal index

Signal	Meaning
`aEdge`	`lineA` changed this clock.
`bitLevel`	Coded level of the bit being sent.
`bitValue`	Decoded value at the current strobe.
`checkPoint`	Moment to compare the read-back pad.
`clockHigh`	Generated clock is really high (always true with push-pull).
`continueNext`	Next entry follows with no gap.
`dataBitEnded`	The bit just sent was data, not a stuff bit.
`dataValid`	A strobe that carries data (not SE0 or SE1).
`dropBit`, `goodBit`	This received bit is a stuff bit / a data bit.
`dropCount`	Entries thrown away by a cancel still to be reported as done.
`endOfBit`	The bit being sent is over.
`endPending`	A frame mark or halt is waiting for its end-of-frame action.
`entryDone`	The entry being sent has no bits (and no stuff bit) left.
`genTick`	Tick from a generated clock.
`holdLevel`	Last level sent, held between entries of a packet.

Signal	Meaning
`i2cBusy`	I²C controller has sent START and not yet STOP.
`idleHold`, `idleLevel`	Whether to hold the last level, and the level used when not sending a bit.
`lastSample`	Previous sample, for NRZI.
`locked`	Manchester decoder has a recent middle edge (or, with `boundary`, has seen the first edge after idle).
`boundary`	`BND`: first edge after idle is a bit boundary.
`driveNow`	"Push-pull while sending" output is driving now.
`nextValid`, `nextLen`, `nextData`	The waiting entry.
`nrziLevel`	Current NRZI line level for transmit.
`packetActive`	Inside a transmitted packet.
`phaseB`	Second half of a generated clock period.
`phaseAClock`	Clock level in phase A (idle level; low for I²C).
`runCount`, `runValue`, `skipNext`	Receive stuffing state.
`rxBit`, `rxTick`, `rxStrobe`	Registered decoded bit, its tick, and the raw sample strobe.
`rxCount`	Position inside the received bit.
`sampleLeading`	Chain samples on the edge in the middle of a generated bit.
`sendBit`	Bit being sent (data or stuff).
`shiftData`, `shiftLeft`	Entry being shifted out and bits left.
`shiftEdge`	SYNC edge on which the output changes.
`startWait`	I²C target waiting for SCL to fall before a new bit.
`step`	Step inside START, Sr, STOP or end of packet.
`strobe`	Sample point of the receive timer (or accepted Manchester edge).
`stuffing`, `stuffBit`	The bit being sent is a stuff bit, and its value.
`txBegin`	A new packet's first entry is loading.
`txCount`	Position inside the bit or step being sent.
`txRun`, `txRunValue`	Transmit stuffing run.
`waitAligned`, `waitBits`	Wait entry state.
`willStuff`	A stuff bit must follow the bit just sent.